High Value Asset/Inventory Risk Oracle - A Hardware-Anchored Assessment System

Sanjeev Jha [253471] , Jasreen Bagga [249403]

Otto-von-Guericke-University

M.Sc in Financial Economics

Scientific Project in FinTech and Blockchain Innovations

March 01, 2026

Cloud link for python file

1. Introduction: Bridging the Gap Between Physical Assets and Digital Records

The modern financial industry has become incredibly efficient at tracking and moving digital money, allowing institutions to process loans and secure transactions in a matter of seconds. However, a major vulnerability remains in the traditional way we manage the real-world, physical assets that back these financial agreements-like using warehouse inventory, precious metals, or agricultural goods as collateral for a loan. The problem lies entirely in the physical world's reliance on manual human intervention and periodic inspections. In traditional scenarios, a human auditor verifies that a warehouse holds 1,000 kilograms of copper, and that static snapshot in time is recorded as a fact on a balance sheet. But physical reality is fluid. Because human audits only happen intermittently, the actual inventory can be tampered with or removed immediately after the auditor leaves the building. The financial record might remain perfectly intact, but the real-world asset it represents could already be gone, leaving lenders completely exposed to fraud until the next scheduled check.

This project was built to solve this exact problem. We have developed an "Oracle"-a bridge that connects physical reality to a digital blockchain. Specifically, we created a hardware-anchored risk assessment system using a Raspberry Pi, a precision weight sensor, and a camera. Instead of relying on a human to manually type inventory updates into a computer, our system physically weighs the collateral in real-time. If the weight drops without authorization, the system instantly snaps a photo for visual proof and automatically alerts an Ethereum smart contract.

1.2. The Flaw of Traditional Audits: Why We Need "Middle" Intervention

To understand why we built this system, we have to look at how asset-based lending works today. Currently, if a company wants to borrow millions of dollars, they might pledge a warehouse full of valuable metal as collateral. To make sure the metal is actually there, the lender sends a human auditor to the warehouse to check the inventory. The critical flaw in this system is that audits only happen periodically-maybe once a month, or once a quarter. This creates a massive "blind spot" or vulnerability window between audits. Traditional management relies heavily on these intermittent human checks, which means the system only provides intervention at the very end of a timeline. If a thief or a fraudulent borrower secretly removes the metal on Day 2, the lender won't find out until the auditor arrives on Day 90. By the time the "end" intervention happens, the collateral is already gone, the money is lost, and the damage is done.

Our project is designed to shift this timeline. We don't want to find out about a theft at the end of the month; we want intervention in the middle of the event. By using continuous IoT (Internet of Things) monitoring, our system acts as a 24/7 digital security guard. The moment the weight on the scale drops below a certain threshold (like 75%, 50%, or 25% of the expected reserve), the system immediately triggers a digital alert. This real-time, middle-intervention approach stops scams while they are happening, rather than just reporting on them after the fact.

2. Case Studies in Collateral Fraud: The Cost of Information Asymmetry

The dangers of relying on paper receipts and delayed human audits are not just theoretical. There are massive, real-world examples where the lack of real-time monitoring led to billions of dollars in losses because lenders and borrowers suffered from "information asymmetry"-meaning the borrower knew the warehouse was empty, but the lender thought it was full.

2.1 The Qingdao Metals Scandal (China, 2014)

The most prominent global example of this vulnerability is the Qingdao Metals Scandal in China. In the port city of Qingdao, a trading company managed to borrow billions of dollars from multiple major international banks. They used large stockpiles of copper and aluminum stored in the port's warehouses as collateral for these loans. However, because the banks relied on paper warehouse receipts and occasional human audits, the trading company was able to commit a massive fraud known as "rehypothecation." They forged the warehouse receipts and used the exact same pile of metal to secure multiple different loans from different banks. When the fraud was finally uncovered, the banks realized that the paper receipts claimed there was vastly more metal than what physically existed in the warehouse. The fraud went unnoticed for so long specifically because there was no continuous, automated monitoring tying the physical weight of the metal to a digital, unchangeable record.

2.2 The NSEL Scam (India, 2013)

A very similar and devastating example occurred in India with the collapse of the National Spot Exchange Limited (NSEL), which resulted in a massive Rs. 5,600 crore financial catastrophe. The exchange was designed to let people trade agricultural commodities and metals, all of which were supposed to be safely stored in designated warehouses. The entire system relied on pieces of paper that acted as the title to the physical stock. However, because nobody was continuously verifying if the physical goods actually existed, bad actors exploited the system. They colluded to generate fake warehouse receipts for commodities that simply were not there. When regulators investigated, they discovered that the warehouses were either completely empty or the physical stock was non-existent. If a hardware oracle had been installed in those warehouses, the sensors would have

immediately detected that the weight was missing, triggering an instant digital alert before the fraud could snowball.

2.3 The LME Nickel Fraud (2023)

A more recent example occurred in 2023 within the London Metal Exchange (LME) network, highlighting a more sophisticated method of deception. A major commodities trading firm purchased what they believed were 54 metric tons of high-grade nickel stored in a warehouse in Rotterdam. When the bags were finally opened and inspected, the auditors found that the bags were filled with useless stones rather than nickel. This case highlights another layer of the problem: a simple weight sensor might be fooled if a fraudster replaces metal with stones of the exact same weight. This is exactly why our hardware oracle was designed to include a camera. If a criminal attempts to tamper with the collateral by swapping materials, the weight sensor will register the movement and the micro-fluctuations during the swap, triggering the camera to capture a real-time photo. This provides an immutable "proof of reserve".

2.4. Why We Built This: A Blueprint for Trustless Custody

Seeing the massive failures in cases like Qingdao, it became clear that the financial industry needs a better way to monitor high-value assets. We built this project because the current system requires too much human trust, and human trust is easily exploited. Our hardware-anchored assessment system replaces subjective human oversight with deterministic smart contract logic. Here is how our system directly solves the problems seen in the case studies:

- Replacing Paper with Blockchain: Instead of paper warehouse receipts that can be easily forged(as seen in Qingdao and NSEL), our system logs the initial "Pledged Baseline" weight directly onto an Ethereum smart contract. Once it is on the blockchain, the record cannot be secretly altered or duplicated.
- Continuous IoT Monitoring: Instead of waiting for a human auditor, our Raspberry Pi and weight sensor monitor the collateral mass continuously. This ensures we catch any unauthorized removal or tampering instantly.

- Real-Time Visual Proof: To prevent situations where collateral is secertly swapped for fake goods (like the stones in the LME nickel scandal), our system integrates visual evidence. The moment the weight changes or fluctuates suspiciously, the camera takes a photo. This photo and the weight data are linked together cryptographically, creating a secure proof that any lender can audit at any time.
- Automated Escalation: We completely remove the delay in reporting. Our system evaluates the live weight ratio directly on the blockchain. If the collateral drops below safe levels, the smart contract automatically changes its state from 'ACTIVE' to 'FLAGGED', and immediately sends an alert to a web dashboard.

By building this system, we aim to provide a blueprint for trustless custody. We want to show that by combining IoT hardware with blockchain smart contracts, we can drastically reduce monitoring costs, eliminate the vulnerability windows where fraud happens, and ensure that the digital record always matches the physical reality of the vault.

3. The Oracle Problem in Blockchain Environments

3.1 Why Blockchains Are "Blind" to the Real World

To understand why our project is necessary, we have to look at how blockchains actually work. Blockchains like Ethereum are incredibly secure because every computer on the network has to agree on the exact same math to process a transaction. Because of this strict rule, a smart contract cannot simply connect to the internet to check a real-world fact. If a smart contract tried to check a website or a sensor, a slight delay or a temporary internet outage could cause different computers to get different answers, and the entire network would crash. Therefore, a blockchain is essentially "blind" to anything outside of its own digital ledger.

3.2 Defining the Oracle Problem

Because the blockchain cannot see the physical world, it relies on a bridge called an "oracle". An oracle is just a piece of software or hardware that fetches data from the real world and feeds it into the smart contract. However, this creates a massive security flaw known as the "Oracle Problem". A blockchain can mathematically guarantee that data cannot be changed after it is saved, but it has no way of knowing if the data was true before

it was saved. If our weight sensor is tricked into telling the blockchain that an empty warehouse is actually full of gold, the smart contract will blindly accept that lie as a fact. Our entire system is only as secure as the data feed itself.

3.3 Why We Need Hardware Oracles, Not Just Software

Most blockchain projects try to solve this by using software oracles. These are programs that check multiple digital sources (like checking the price of Bitcoin on three different websites) and average them out to ensure the data is accurate. But software oracles only work for digital data. You cannot send an internet request to physically weigh a bar of copper sitting in a warehouse. To bridge physical assets into digital finance, we had to build a Hardware Oracle. By using a Raspberry Pi connected to physical sensors, our system actively measures the real world (like mass and movement) and translates that into cryptographic numbers the blockchain can understand.

3.4 The Real-World Risks of Physical Sensors

Using physical hardware introduces real-world risks that software doesn't have to deal with. When relying on hardware at the "edge" of the network, we had to account for a few major threats:

- Faking the Data (Spoofing): A hacker could try to set up a fake sensor to send fake weights to the blockchain. To stop this, our Raspberry Pi acts as a secure gateway, digitally "signing" every piece of data so the smart contract knows exactly which physical machine sent it.
- Sensor Wear and Tear: Physical metal scales degrade over time. Heavy weights sitting on a scale for months can cause the sensor to slowly drift and give slightly inaccurate readings, even if no metal was stolen.
- Physical Tampering: The hardest problem to solve is physical sabotage. A smart thief could physically cut the wires connecting our scale to the Raspberry Pi and feed fake electrical signals into it. This is exactly why our camera takes a photo when it detects suspicious movement, adding a layer of visual proof that raw data alone cannot provide.

3.5 Why Machines Are Better Than Human Audits

Despite the physical risks of using sensors, this system is still vastly superior to traditional human audits. A human auditor can be bribed, threatened, or simply make a mistake. A hardware oracle, however, follows strict mathematical rules. By ensuring that the actual financial math (calculating if the loan is safe or risky) happens on the unchangeable blockchain, we remove human bias and corruption entirely from the audit process.

4. Edge Computing Integration: The Hardware-to-Blockchain Bridge

4.1 Why We Don't Send Everything to the Blockchain

When building this project, we quickly realized that we couldn't just stream live weight data to the blockchain every second. Public blockchains charge "gas fees" (transaction costs) every time you update their records. If our sensor updated the blockchain every time the weight fluctuated by a few grams, the system would run out of money in hours. To fix this, we used "Edge Computing". Our Raspberry Pi acts as a smart filter at the edge of the network. It processes all the continuous weight sampling locally. It only spends money to talk to the blockchain when something significant actually happens-like an unauthorized removal of collateral.

4.2 Translating Physical Weight into Digital Numbers

To actually read the weight of the collateral, our system relies on a 5 kg Load Cell. A load cell is essentially a piece of metal with wires that change their electrical resistance when the metal bends under weight. However, a Raspberry Pi only understands digital code (1s and Os), and the load cell only produces tiny electrical voltages. To bridge this gap, we wired an HX711 chip between them. The HX711 acts as a translator, taking the tiny physical voltage and converting it into a clean digital integer that our Python script can read and calculate into actual grams.

4.3 Filtering Out Warehouse Noise

In a real-world warehouse, the environment is never perfectly still. Heavy machinery driving by or a change in room temperature can cause the scale to vibrate or drift slightly. If our system reacted to every tiny vibration,

it would constantly send false alarms to the lender. To prevent this, we wrote a specific filtering algorithm in our Python code. Instead of just taking one weight reading, the system takes a burst of 20 readings in a fraction of a second. It then sorts those numbers, throws away the highest and lowest 20% (which removes any sudden spikes or errors), and averages the remaining stable numbers. This ensures the weight we report is highly accurate and intentional.

4.4 The 9-Step Risk Oracle Cycle

All of this hardware and software works together in a fully automated loop. As shown in our project poster, we call this the 9-Step Risk Oracle Cycle, and it runs continuously without human intervention:

1. Baseline Establishment: The very first time the collateral is weighed, that number is permanently saved to the blockchain as the expected baseline.
2. Continuous IoT Monitoring: The sensors scan the physical mass 24/7 looking for changes.
3. Cryptographic Signing: If a change happens, the Raspberry Pi digitally signs the data so nobody can intercept and fake it.
4. Oracle Transmission: The system securely sends this signed data to the Ethereum smart contract.
5. Multi-Stage Threshold Detection: The system checks the weight against safety checkpoints (like 75%, 50%, or 25% of the total amount).
6. Sensor-Triggered Visual Capture: The exact moment the weight drops, the camera takes a photo to provide visual proof of what happened.
7. On-Chain Logic Evaluation: The smart contract does the math to see if the loan is still safe based on the new weight.
8. Contract State Transition: If the collateral is missing, the contract changes its official status from “ACTIVE” to “FLAGGED”.
9. Automated Escalation & Alert: The web dashboard lights up, notifying the lender that human intervention is immediately required.

5. Cryptographic Primitives and Off-Chain Middleware

5.1 The Brain of the Operation: Our Python Middleware

For our hardware to actually be useful, the Raspberry Pi needs a "brain" to process what the weight sensor is feeling. We wrote a Python script (App.py) that acts as this brain. Instead of just taking one reading and turning off, this script runs in an endless loop in the background, constantly checking the physical weight of the collateral. We programmed a very specific rule into this loop to prevent false alarms: the system only cares about significant drops in weight. If someone tries to cheat by adding extra weight (like throwing a heavy rock on the scale), the Python script simply ignores the increase. It only reacts when weight is removed, ensuring the smart contract only triggers a penalty when the collateral is actually in danger.

5.2 Packaging the Evidence

When the Python script confirms that a chunk of weight is actually missing, it needs to build an official incident report. Just sending a number like "500 grams" to the blockchain isn't enough evidence. The moment the weight drops, the script triggers the Raspberry Pi camera to take a real-time photo of the scale. It then bundles the new weight reading, the exact timestamp, and the file path of the photo into a neat, organized data package called a JSON file. This JSON file acts as our immutable "proof of reserve," combining the physical measurement with visual evidence.

5.3 Securing the Data with SHA-256 Hashing

Here is where we ran into a major engineering hurdle: blockchains are incredibly expensive places to store data. Uploading a high-resolution JPEG image and a text file directly to the Ethereum network would cost way too much in "gas fees" (transaction costs). To solve this, we use a cryptographic tool called SHA-256. This algorithm takes our entire JSON proof file (including the photo data) and scrambles it into a fixed 32-byte string of letters and numbers called a "hash". The beauty of SHA-256 is that it works in one direction. If a hacker tries to edit the JSON file later-even changing a single pixel in the photo or changing the weight by one gram-the hash will completely change. By only uploading this tiny 32-byte hash

to the blockchain, we save a massive amount of money on gas fees while still permanently locking the evidence to the digital ledger.

5.4 Sending it to the Blockchain

Finally, the Python script has to physically send this data to the blockchain. We use a library called Web3.py to connect our Raspberry Pi to the Sepolia test network. To prove that the data is legitimately coming from our warehouse and not from a hacker pretending to be us, the Raspberry Pi uses a private, secret key to digitally "sign" the transaction. Once signed, this alert is broadcasted out to the network, where Ethereum miners permanently record the missing weight into the blockchain. The private key has been withheld from the code to ensure compliance with data privacy constraints

6. Smart Contract Architecture and Deterministic State Transitions

6.1 The Smart Contract: Our Unchangeable Judge

While the Python script acts as the brain collecting the data, it is ultimately just a messenger. The real power of this project lies in the Ethereum smart contract, which acts as the unchangeable judge. Written in a language called Solidity and deployed on the Sepolia network, this contract holds the official, permanent record of the loan. It safely stores the starting weight (the baseline) and the current weight reported by the sensor. Because this contract lives on the blockchain, no bank, auditor, or borrower can secretly log in and alter the rules.

6.2 The reportWeight Function

When the Raspberry Pi detects a theft, it calls a specific piece of code inside our smart contract named the reportWeight function. Because every line of code executed on Ethereum costs money, we designed this function to do four things instantly and efficiently:

1. It updates the official ledger with the newly reported weight.
2. It calculates how healthy the collateral is right now.
3. It updates the risk status of the loan if necessary.

4. It broadcasts an alert to the rest of the network.

6.3 Doing the Math On-Chain

We made a very specific design choice here: we forced the smart contract to do the math, not the Raspberry Pi. The smart contract takes the current weight, multiplies it by 100, and divides it by the original pledged weight to get a clean percentage. By doing this calculation publicly "on-chain", we completely remove the need to trust the Python script. The hardware merely reports the raw physical fact, and the blockchain strictly enforces the financial reality.

6.4 Automating the Risk Levels

Once the smart contract calculates that percentage, it automatically updates the health status of the loan using a hardcoded set of rules:

- If the weight is above 75% of the original amount, the state stays ACTIVE.
- If it drops between 50% and 75%, it escalates to FLAGGED_L1.
- If it drops between 25% and 50%, it escalates to FLAGGED_L2.
- If it drops below 25%, it hits the critical FLAGGED_L3 state.

In traditional finance, a human credit analyst would have to read a report and decide if the borrower is in default. Our system automates this entire escalation process through code, leaving zero room for human hesitation or bias.

6.5 Broadcasting to the Web Dashboard

Blockchains are amazing at storing data securely, but they are notoriously bad at showing that data to regular users. To fix this, the last thing our reportWeight function does is emit a digital signal called a WeightReported event. This acts like a flare shot up into the sky. Emitting an event is much cheaper than saving data permanently, and it allows standard Web2 websites (like the dashboard we built for the lender) to easily "listen" to the blockchain. The moment the smart contract changes the status to FLAGGED, the dashboard picks up the event and instantly updates the screen with a warning alert.

7. UI/UX Front-End Design for Web3 Stakeholders

7.1 Making Blockchain Data Human-Readable

Blockchains are incredible at securely storing data, but their native output is basically unreadable for normal humans. If you look directly at an Ethereum transaction, all you see is a string of hexadecimal letters and numbers. A lender or risk manager cannot look at raw machine code and make a split-second financial decision. To solve this, we built a web dashboard that translates this complex blockchain data into a clean, easy-to-read interface. It takes the raw numbers and visualizes the pledged weight, the live sensor weight, the overall collateral ratio, and the current risk state in real-time.

7.3 The "Dual-State" Problem

Our presentation layer is powered directly by the Raspberry Pi using a lightweight Python framework called Flask. We ran into an interesting design challenge: because we only send significant weight drops to the blockchain to save money on fees, there is often a slight difference between what the physical scale reads and what the blockchain officially records. To be completely transparent, our dashboard shows a "Dual-State" view:

1. Live Sensor Weight (Local): This shows exactly what the physical scale is feeling right this second, giving the lender an absolute real-time view.
2. On-Chain Ratio (Official): This is the official, globally recognized truth locked into the Ethereum network.

We also added a "Local On - Chain" card that calculates what the blockchain would say if we pushed the live sensor data right now. This lets lenders foresee potential drops in their collateral health before it officially hits the blockchain.

7.4 Tracking the Health of the Loan

To make the status of the loan immediately obvious, we built a dynamic progress bar. As the weight fluctuates, the bar changes color to indicate severity:

- Safe: The collateral is above 75%.

- Warning: The collateral has dropped between 25% and 75%, indicating an active bleed.
- Critical: The collateral has dropped below 25%, meaning the vault is almost empty.

8. System Limitations and Real-World Constraints

8.1 The Physics of Load Cells and Averaging Delays

When building a bridge between the physical and digital worlds, we have to deal with the laws of physics. Our system uses a load cell, which calculates weight by measuring microscopic bends in a piece of metal. Because this is a physical measurement, it is not perfectly instantaneous like a digital API call. To get a highly accurate number, our Python script takes a burst of raw readings and calculates an average to filter out errors. While this makes the data much more reliable, it also means the sensor is inherently slightly "slow." There is a minor time delay between the exact moment the weight is removed and the moment the software confidently confirms the final average weight. For our project's scope, a delay of a few seconds is perfectly acceptable, but it is a physical limitation of the hardware.

8.2 Sensitivity to Base Movement and Vibrations

Another reality of physical hardware is that it feels everything. If the base of the weight sensor moves, or if the floor vibrates, the load cell will register those micro-movements as changes in weight. In a real-world warehouse, forklifts drive by, heavy machinery operates, and pallets are dropped. These environmental vibrations can cause the raw weight data to fluctuate wildly for a few seconds. While our software algorithm is designed to ignore these temporary spikes and only react to permanent weight drops, extreme or constant vibrations in a busy industrial setting could temporarily delay the system from getting a clean, stable reading.

8.3 Environmental Sensor Drift

We also have to account for how the environment affects metal and electronics over long periods. If hundreds of kilograms of metal sit on a load cell for several months, the strain gauges can experience "sensor creep"- meaning the metal permanently deforms just a tiny bit. Additionally,

significant changes in warehouse temperature can alter the electrical resistance inside our HX711 analog-to-digital converter. Over a long enough timeline, this can cause the baseline reading to slowly drift away from the true weight. In a permanent, multi-year deployment, the system would occasionally need to be quickly recalibrated to ensure the starting zero-point remains perfectly accurate.

8.4 The Scope of the Smart Contract Trigger

The primary goal of this project was to successfully build an automated, trustless audit system. We accomplished this by linking the physical sensor to an Ethereum smart contract that evaluates the health ratio and deterministically triggers a state change (from ACTIVE to FLAGGED). Our system perfectly executes this scope: it monitors, verifies, and permanently logs the alert on the blockchain. We do not view it as a limitation that the system stops at the flagging stage. The purpose of this oracle is to act as the ultimate, unhackable alarm system. Once the smart contract flips to the FLAGGED state, it has done its job, permanently alerting the lender that the collateral has been compromised so they can take immediate action.

9. Future Scalability and Enterprise Integration

9.1 Enterprise Storage Integration

For this project, whenever the camera captures visual evidence of a weight drop, we save the image directly into a local folder on the Raspberry Pi. This is the perfect lightweight solution for a proof-of-concept. However, we recognize that a major bank or lending institution would not store millions of dollars worth of audit evidence on a local SD card. In a future commercial rollout, an enterprise client would simply connect our Python data pipeline directly to their own secure cloud storage (like AWS or Google Cloud). Because our system already packages the evidence neatly into a standardized JSON file and generates a cryptographic SHA-256 hash, plugging this system into a bank's existing cloud infrastructure would be a seamless upgrade.

9.2 Scaling with Layer 2 Networks

Currently on the Sepolia Testnet, our architecture is highly cost-optimized because we strictly separate read operations from state-changing transactions. While our web dashboard can freely query the smart contract's state via RPC calls without incurring costs, the Raspberry Pi only executes a state-changing transaction (which costs "gas fees") when a significant weight drop triggers an alarm. Otherwise, the system avoids writing to the blockchain, costing nothing. However, enterprise banks cannot rely solely on this "report by exception" model, as a broken or offline sensor is also silent. Commercial compliance requires daily "heartbeat" updates-actual state-changing transactions logged to the blockchain to legally verify sensor uptime and exact collateral weight. Logging daily heartbeats for thousands of pallets would generate astronomical gas fees on the Ethereum Mainnet. To scale affordably, the system must deploy to a "Layer 2" network like Arbitrum or Optimism. By bundling thousands of transactions together, Layer 2 networks reduce fees to pennies while maintaining strict base-layer security for massive commercial warehouses.

9.3 Multi-Sensor Warehouse Arrays

Our project proves that a single edge device can successfully monitor a vault and securely report to a smart contract. To scale this up for a massive, commercial warehouse, the architecture would expand from a single Raspberry Pi into a "Multi-Sensor Array." Instead of one sensor under a pallet, a bank could install dozens of weight sensors across the floor, all communicating with the same smart contract. The contract could be upgraded to require a consensus-for example, requiring at least two neighboring sensors to confirm a weight drop before sounding the alarm. This would provide incredible redundancy and ensure the system never goes offline even if one piece of hardware is accidentally damaged.

9.4 Building on the Flagging System

Because we successfully built the foundational smart contract that securely flags compromised assets, future developers can easily stack new financial tools on top of our work. Now that the FLAGGED state is reliably triggered by real-world physical data, a DeFi (Decentralized Finance) protocol could build automated liquidation bots that "listen" for our smart contract's alarm. The moment our system switches to FLAGGED_L3, those

bots could automatically sell off the borrower's digital assets to protect the lender. Our project provides the secure physical-to-digital bridge that makes those advanced financial automations possible.

References

[1] Engineer, D., & Singh, C. (2013). The NSEL scam: Bridging the regulatory hiatus and its fallout on the corporate sector in India. *NLIU Law Review, 5*(2), 301–327.

[2] International Monetary Fund. (2013). *India: Financial system stability assessment* (Country Report No. 13/8). `https://www.imf.org/external/pubs/ft/scr/2013/cr1308.pdf`

[3] Lamberti, L., & Schimkat, C. (2024). Supply chain integrity and the role of IoT oracles: Lessons from the 2023 LME nickel fraud. *International Journal of Logistics Management, 35*(1), 112–130.

[4] Ng, A. K., & Wong, J. T. (2015). The Qingdao port metal scandal: Implications for commodity financing and risk management in China. *Journal of Financial Regulation and Compliance, 23*(3), 254–268.

Declarations

Declaration on the Use of Artificial Intelligence

This report was prepared with the support of artificial intelligence tools used solely for research assistance, learning support, and methodological clarification. These tools aided in refining understanding, organizing ideas, and improving clarity of expression.

All analysis, interpretation, conclusions, and final content are the author's own work. No AI system was used to generate original research data, conduct experiments, or replace independent critical judgment. The author assumes full responsibility for the accuracy, integrity, and originality of the work presented.